Криптографія
Комп’ютерний практикум №2

Криптоаналіз шифру Віженера

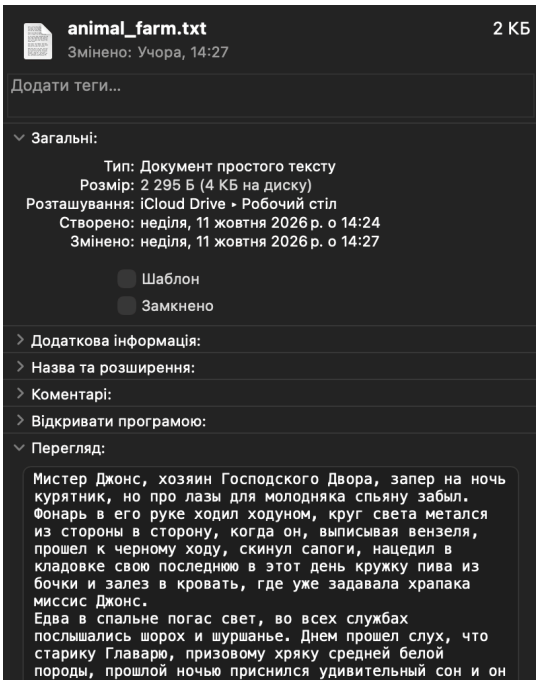
Виконали:
студенти групи ФБ-41
Александрова М. та Проценко Д.

Мета: засвоєння методів частотного криптоаналізу. Здобуття навичок роботи та аналізу потокових шифрів гамування адитивного типу на прикладі шифру Віженера.

Варіант: 5

Хід роботи

1. Дослідження шифру Віженера
Самостійно підібрати текст російською мовою обсягом 2–3 КБ та підготувати його відповідно до алфавіту, прийнятого в роботі.



Для роботи ми обрали фрагмент тексту “Скотный двор”.

Функція для підготовки тексту (приведено до нижнього регістру, без пробілів, розділових знаків та інших символів, що не належать до алфавіту, «ё» замінюється на «е»):

```python
# підготовка тексту
def clean_text(file_path):
    f = open(file_path, "r", encoding="utf-8")
    raw = f.read().lower()
    f.close()

    raw = raw.replace("ё", "е")
    text = re.sub(r"[^а-я]", "", raw)  # залишаємо тільки літери
    return text
```

В роботі ми використовуємо алфавіт "абвгдежзийклмнопрстуфхцчшщъыьэюя" (32 літери).

Реалізувати програмні засоби, необхідні для виконання завдань практикуму. Зашифрувати один і той самий відкритий текст шифром Віженера ключами довжини r = 2, 3, 4, 5, а також одним ключем довжини 10–20 символів.

Функція для шифрування тексту шифром Віженера.

```python
# шифрування шифром Віженера
def vigenere_encrypt(text, key):
    result = ""
    for i in range(len(text)):
        x = alphabet.index(text[i])
        k = alphabet.index(key[i % len(key)])
        y = (x + k) % m
        result += alphabet[y]
    return result
```

Кожна літера ВТ шифрується за такою формулою:
$y_i = (x_i + k_{i \bmod r}) \bmod m,$
де x_i це номер і-ї літери відкритого тексту, $k_{i \bmod r}$ це номер літери ключа, y_i це номер і-ї літери шифртексту, r це довжина ключа.

Ідея шифрування полягає в тому, що кожній літері алфавіту ставиться у відповідність її номер: а - 0, б - 1, …, я - 31 (всього m = 32). Шифрування зводиться до додавання номера літери відкритого тексту та номера відповідної літери ключа за модулем 32. Ключ підписується під відкритим текстом і повторюється стільки разів, скільки потрібно.

Для відкритого тексту та всіх одержаних шифртекстів обчислити індекси відповідності, порівняти їх значення та пояснити одержані результати.

Індекс відповідності - це статистичний показник, що визначає ймовірність того, що дві випадково обрані літери із зашифрованого тексту виявляться однаковими.

Він обчислюється за формулою:

$I(Y) = \sum \frac{N_t (N_t - 1)}{n(n-1)}$, де N_t - кількість появ букви t у шифртексті Y, n - довжина тексту

```python
# індекс відповідності
def index_of_coincidence(text):
    n = len(text)
    if n < 2:
        return 0
    counts = Counter(text)
    s = 0
    for c in counts:
        s += counts[c] * (counts[c] - 1)
    return s / (n * (n - 1))
```

Частоти літер підраховуються за допомогою класу Counter. Для кожної літери обчислюється величина N_t(N_t − 1), отримані значення сумуються, і сума ділиться на n(n − 1).

Для шифрування ми використовували 5 ключів різної довжини (2, 3, 4, 5 та 12 символів):

```
keys = ["да", "лес", "море", "весна", "криптография"]
for key in keys:
    ct = vigenere_encrypt(text, key)
    print(f"r = {len(key)}, ключ: {key}")
    print("шифртекст:", ct[:60])
    print(f"I = {index_of_coincidence(ct):.5f}\n")
print()
```

Результати:

```
margarita@Air-Margarita кп2 % python3 lab2.py
Довжина тексту: 1030
Відкритий текст: мистерджонсхозяингосподскогодвораза пернаночькурятникнопролаз
I відкритого тексту: 0.05389

r = 2, ключ: да
шифртекст: рихтйрижтнххтзгисгтсуоисоозоивтрдздпйрсасоыьоуфяцнмксоуртлдз
I = 0.04390

r = 3, ключ: лес
шифртекст: чнвэкбпляшцжщмрутфщцащйвхуфщйущхстеархюлтявбыюхрэтщхтяъхяцеш
I = 0.03891

r = 4, ключ: море
шифртекст: шцбчсюфлъыбъъхпнщсюцыьфццьууррюхмхрфсюэещьзбцбадюышпщьяхъщрм
I = 0.03721

r = 5, ключ: весна
шифртекст: онвяетйчынуъяфяктфыссухюкриясврхсфаскбъапуийкххрянкпюыптуьнз
I = 0.03496

r = 12, ключ: криптография
шифртекст: цшщбчюзцобщфшчзчяссбпвмрфюлэцрсааыиопахпяьъмкзшюьэрщяьтаояиж
I = 0.03314
```

Текст	Довжина ключа	Ключ	Індекс відповідності
ВТ	-	-	0.05389
ШТ 1	2	да	0.04390
ШТ 2	3	лес	0.03891
ШТ 3	4	море	0.03721
ШТ 4	5	весна	0.03496
ШТ 5	12	криптография	0.03314

За результатами експерименту видно, що зі збільшенням довжини ключа індекс загалом спадає: від 0.04390 при r = 2 до 0.03314 при r = 12.

Для рівноймовірного алфавіту з 32 символів:
$I_0 = \frac{1}{32} = 0.03125$
Тобто чим довший ключ, тим ближче індекс до значення для випадкового тексту (1/32). Це пояснюється тим, що шифр Віженера з ключем довжини r фактично складається з r шифрів Цезаря з різними зсувами, які застосовуються до літер відкритого тексту по черзі. Кожен окремий шифр Цезаря зберігає частоти літер (змінюється лише те, якій літері вони належать), але піки частот різних блоків припадають на різні літери. У шифртексті в цілому вони накладаються один на одного, і розподіл літер вирівнюється. Чим довший ключ, тим менша частка пар літер, що належать одному блоку (≈ 1/r), і тим більша частка пар літер із різних блоків, які збігаються лише з імовірністю ≈ 1/32. Тому індекс відповідності прямує до значення для рівноймовірного алфавіту.

Індекс відповідності для відкритого тексту - 0.05389, це найвищий показник, бо природна мова має нерівномірний розподіл літер.

2. Криптоаналіз шифртексту
Для наданого викладачем шифртексту (згідно з номером варіанта) виконати криптоаналіз шифру Віженера.
Довжину ключа визначати статистичними методами, описаними в розділі «Необхідні теоретичні відомості». При пошуку періоду перевірити можливі значення r щонайменше до 30. У звіті навести результати, на підставі яких було обрано довжину ключа. Недостатньо вказати лише знайдене значення r. Дослідити декілька альтернативних кандидатів на довжину ключа. Обґрунтувати вибір кандидатів та пояснити отримані результати.

Для визначення довжини ключа r було проаналізовано шифртекст двома статистичними методами для значень $r \in [1, 30]$:

1. Метод середнього індексу відповідності (IC_{avg}) для r стовпчиків.

```
def get_avg_ic(text, r):
    columns_ic = [calc_ic(text[i::r]) for i in range(r)]
    return sum(columns_ic) / r
```

2. Метод автокореляції (статистика збігів D_r).

```
def get_d_r(text, r):
    return sum(1 for i in range(len(text) - r) if text[i] == text[i + r])
```

Загальний індекс відповідності всього шифртексту становить $IC_{\text{тексту}} \approx 0.0353$, що близько до теоретичного індексу рівномірного випадкового розподілу для 32-буквеного алфавіту ($\frac{1}{32} \approx 0.03125$).

```
Індекс відповідності нашого  тексту: 0.03532444245066751
r    | Середній ІК     | D_r
---------------------------------
1    | 0.03532         | 182
2    | 0.03710         | 211
3    | 0.03535         | 192
4    | 0.03979         | 180
5    | 0.03544         | 192
6    | 0.03705         | 206
7    | 0.03522         | 173
8    | 0.04491         | 189
9    | 0.03545         | 180
10   | 0.03710         | 211
11   | 0.03506         | 165
12   | 0.03979         | 196
13   | 0.03551         | 214
14   | 0.03709         | 198
15   | 0.03538         | 198
16   | 0.05540         | 302        <--- Скоріше за все це наш період
17   | 0.03552         | 182
18   | 0.03705         | 216
19   | 0.03532         | 208
20   | 0.03980         | 207
21   | 0.03506         | 172
22   | 0.03688         | 169
```

```
24   | 0.04486         | 173
25   | 0.03532         | 187
26   | 0.03731         | 195
27   | 0.03525         | 192
28   | 0.03969         | 175
29   | 0.03558         | 214
30   | 0.03693         | 215
```

Значення $r = 16$ є основними і єдиним підтвердженим кандидатом на довжину ключа з наступних причин:

> **Збіг із мовною статистикою:** При $r = 16$ середній індекс відповідності сягає 0.05540. Це відповідає теоретичному індексу відповідності відкритого тексту російської мови (≈ 0.0553). Це свідчить про те, що кожен із 16 стовпчиків зашифрований одноалфавітним зсувом

Виражений пік автокореляції: Статистика збігів D_{16} показує різкий сплеск до 302 (при фоновому рівні шуму $\sim 180 - 210$). Символи, розміщені на відстані 16 позицій один від одного, зашифровані однією і тією ж буквою ключа.

У ході дослідження виявлено декілька альтернативних кандидатів з підвищеними показниками:

Кандидати $r = 8$ та $r = 24$ ($IC \approx 0.0449$):

- *Пояснення:* Число 8 є дільником 16 ($16 = 8 \times 2$), а 24 має з 16 спільний дільник 8. При розбитті тексту на 8 стовпчиків кожен стовпчик складається з двох «правильних» стовпчиків (для $r = 16$). Через часткове не збереження структури періодичності IC піднімається вище фонового значення, але не досягає мовної норми (0.055).
- *Чому відхилено:* Значення $D_8 = 189$ та $D_{24} = 173$ залишаються на рівні випадкового шуму і не дають сплеску.

Кандидат $r = 4$ ($IC \approx 0.0398$):

- *Пояснення:* Число 4 також є дільником 16 ($16 = 4 \times 4$). Це створює слабкий гармонійний сплеск IC, але значно нижчий за $r = 8$ та $r = 16$.
- *Чому відхилено:* Значення IC занадто далеко від 0.055, а $D_4 = 180$ відповідає шумному фону.

Висновок: Альтернативні піки на $r = 4, 8, 24$ є математичним наслідком кратності справжньому періоду $r = 16$

3. Відновлення ключа та відкритого тексту

Після визначення довжини ключа відновити ключ за методикою, описаною в теоретичних відомостях до роботи.
У звіті навести проміжні статистичні результати, достатні для обґрунтування вибору символів ключа.

Отже, ми встановили, що довжина ключа = 16. За методикою, описаною в теоретичних відомостях, потрібно розбити ШТ на 16 блоків:

$Y_0 = y_0, y_{16}, y_{32}$ …
$Y_1 = y_1, y_{17}, y_{33}$ …
$Y_2 = y_2, y_{18}, y_{34}$ …
…
$Y_{15} = y_{15}, y_{31}, y_{47}$ …,

де y_0, y_{16}, y_{32} … - це літери одного блоку, зашифровані однією і тією ж літерою ключа

Виходить, що кожен блок це шифр Цезаря.
Визначимо 5 найчастіших символів кожного блоку:

```
margarita@Air-Margarita кп2 % python3 lab2_3.py
Довжина шифртексту: 5614

Блок 0 (довжина 351 ):
т – 35; й – 29; с – 25; п – 24; х – 23
Блок 1 (довжина 351 ):
у – 40; т – 29; н – 27; е – 25; к – 24
Блок 2 (довжина 351 ):
ш – 32; р – 32; щ – 28; у – 28; л – 25
Блок 3 (довжина 351 ):
у – 32; ц – 29; ь – 29; о – 28; ы – 24
Блок 4 (довжина 351 ):
щ – 41; р – 32; л – 30; у – 24; ш – 22
Блок 5 (довжина 351 ):
ц – 41; н – 35; р – 34; х – 31; и – 24
Блок 6 (довжина 351 ):
я – 34; ц – 34; ю – 32; с – 31; в – 24
Блок 7 (довжина 351 ):
ь – 52; у – 40; ц – 24; о – 22; щ – 19
```

```
Блок 8 (довжина 351 ):
п – 31; й – 29; ж – 29; у – 24; б – 23
Блок 9 (довжина 351 ):
ь – 38; у – 37; о – 31; ы – 27; ц – 25
Блок 10 (довжина 351 ):
ю – 42; б – 32; х – 31; р – 26; ш – 23
Блок 11 (довжина 351 ):
ь – 39; ы – 27; у – 27; о – 24; ц – 24
Блок 12 (довжина 351 ):
ч – 41; а – 35; ъ – 29; э – 27; я – 25
Блок 13 (довжина 351 ):
т – 34; ы – 32; н – 28; ъ – 24; п – 21
Блок 14 (довжина 350 ):
у – 35; к – 33; р – 23; т – 23; е – 22
Блок 15 (довжина 350 ):
ч – 50; с – 32; о – 30; ц – 26; й – 25
margarita@Air-Margarita кп2 %
```

Довжина шифротексту - 5614, націло на 16 не ділиться, тому довжина блоків 0-13 становить 351 символ, а решти - 350 символів.

У більшості блоків помітно виражені найчастиши символи, у блоці 0 це “т” (зустрічається 35 разів), у 1 - “у” (40 разів), 4 - “щ” (41 раз) і тд.

Літеру ключа можна знайти за формулою:
$k = (y^* - x^*) \, mod \, m$, де
y^* – буква, що частіше за всіх зустрічається у блоці
x^* – найімовірніша буква в російській мові (“о”)

```python
# k = (y* - x*) mod m
def get_key_letter(y_star, x_star):
    k = (alphabet.index(y_star) - alphabet.index(x_star)) % m
    return alphabet[k]
```

```
margarita@Air-Margarita кп2 % python3 lab2_3.py
Довжина шифртексту: 5614

Ключ при x* = 'о':
Блок 0 : y* = т -> k = д
Блок 1 : y* = у -> k = е
Блок 2 : y* = ш -> k = к
Блок 3 : y* = у -> k = е
Блок 4 : y* = щ -> k = л
Блок 5 : y* = ц -> k = и
Блок 6 : y* = я -> k = с
Блок 7 : y* = ь -> k = о
Блок 8 : y* = п -> k = б
Блок 9 : y* = ь -> k = о
Блок 10 : y* = ю -> k = р
Блок 11 : y* = ь -> k = о
Блок 12 : y* = ч -> k = й
Блок 13 : y* = т -> k = д
Блок 14 : y* = у -> k = е
Блок 15 : y* = ч -> k = й

Початковий ключ: декелисоборойдей
```

Початковий ключ вийшов: декелисоборойдей
Він не зовсім змістовний. Щоб дізнатись, які саме позиції ключа неправильні, можна вивести розшифрований цим початковим ключем текст по 16 літер в рядку. Візьмемо перші 160 символів, щоб отримати 10 повних рядків.

```
# розшифрування (зворотна дія до розшифрування)
def decrypt(text, key):
    result = ""
    for i in range(len(text)):
        y = alphabet.index(text[i])
        k = alphabet.index(key[i % len(key)])
        result += alphabet[(y - k) % m]
    return result

# розшифровуємо фрагмент тексту початковим ключем, щоб знайти неправильні позиції
plain = decrypt(cipher, key)   # key = "декелисоборойдей"
```

```
Розшифрування по 16 літер у рядку:
пооитноеделоуьль
тусьнасильнолаел
овжуаневоткнобьв
ореьсиэтудовчфьн
огсьстнуюистсцуз
намснаверноефьчш
ечжхгдебытонскыл
овнсрекультущцос
тьрщеждевсегчьси
лижсежелионоъдзм
```

Позиція у ключі дорівнює номеру стовпчика (i mod 16), тому якщо вивести текст рядками по 16 літер, кожен стовпчик відповідає одній літері ключа.

В другому та шостому рядку видно цілі слова “насильно” та “наверное”, отже на позиціях 4-11(якщо рахувати від 0) точно стоять правильні літери.
В першому рядку “пооитноедело” схоже на “понятноедело”, отже, можна припустити, що літери на 0 та 1 позиції також правильні. Якщо поглянути на стовпці 14 та 15, там літери цілком нормальні та поєднуються з наступними літерами з рядків нижче.
Отже, на позиціях 2, 3, 12, 13 точно стоять неправильні літери, значить для цих блоків можна спробувати взяти наступну за ймовірністю появи в російській мові букву “е”.

```
# беремо наступну за імовірністю букву, щоб знайти правильні літери ключа для позицій 2, 3, 12, 13
x_star = "е"  # друга за імовірністю літера мови
problem_blocks = [2, 3, 12, 13]  # блоки, де ключ вийшов неправильно

print()
print("Кандидати при x* = 'е':")
for i in problem_blocks:
    counts = Counter(blocks[i])
    top = counts.most_common(2)   # дві найчастіші літери (для випадків з нічиєю)

    parts = []
    for c, cnt in top:
        k = get_key_letter(c, x_star)
        parts.append(c + "(" + str(cnt) + ") -> " + k)

    print("Блок", i, ":", "; ".join(parts))
```

```
Кандидати при x* = 'е':
Блок 2 : ш(32) -> у; р(32) -> л
Блок 3 : у(32) -> о; ц(29) -> с
Блок 12 : ч(41) -> т; а(35) -> ы
Блок 13 : т(34) -> н; ы(32) -> ц
```

Початковий ключ: декелисоборойдей
На другу та третю позицію ставимо відповідно “л” та “о” та отримуємо слово “дело”
На 12 та 13 - “т” та “н” відповідно, та отримуємо слово “оборотней”

Отже: остаточний ключ: делолисоборотней

Спробуємо розшифрувати фрагмент тексту, щоб впевнитись:

```python
# виправляємо неправильні літери на позиціях 2, 3, 12, 13
fix = {2: "л", 3: "о", 12: "т", 13: "н"}
key_list = list(key)
for i in fix:
    key_list[i] = fix[i]
key = "".join(key_list)
print("Ключ після виправлень:", key)
print(decrypt(cipher, key)[:200])
print("I розшифрованого:", round(index_of_coincidence(plain), 5))
```

```
Ключ після виправлень: делолисоборотней
понятноеделокультурунасильновчеловеканеваткнешьвордусиэтудовольногрустнуюистин
узналинаверноелучшечемгдебытонибыловмирекультурностьпреждевсегоусилиеиежелионо
сызмальстванесделалосьчеловекусвычнымдажевну
```

“Понятное дело, культуру насильно в человека не воткнёшь. В Ордуси эту довольно грустную истину знали, наверное, лучше, чем где бы то ни было в мире. Культурность прежде всего усилие, и ежели оно сызмальства не сделалось человеку свычным, даже…”
Як видно, фрагмент розшифровано коректно. «Дело лис-оборотней» — це фантастичний роман, який написав літературний проєкт Хольм ван Зайчик, а розшифрований фрагмент ймовірно було взято з циклу “Плохих людей нет”.

4. Дослідження впливу помилки у ключі
Змінити один довільний символ правильно відновленого ключа та повторно розшифрувати той самий шифртекст.
До проведення експерименту спрогнозувати, як така зміна вплине на результат розшифрування. Порівняти прогноз із результатом експерименту та пояснити отриманий
ефект.

Теор:Оскільки розшифрування у шифрі Віженера виконується посимвольно, зміна одного символу в ключі довжиною $r = 16$ спотворить лише один із 16 стовпчиків.

Тобто “зіпсується” рівно 6.25% тексту(1\r / 1/16), а решта 93.75% залишиться абсолютно читабельною.

Експеримент:

```
# --- Дослідження впливу помилки у ключі ---
bad_key = "г" + key[1:]
bad_plain = decrypt(cipher, bad_key)

print("\nТекст з помилкою у ключі")
print("Ключ з помилкою:", bad_key)
for start in range(0, 160, r):
    print(bad_plain[start : start + r])
print("I розшифрованого:", round(calc_ic(bad_plain), 5))
```

Модифікуємо ключ и змінюємо першу літеру “д” на “г”

Результат:

```
Текст з помилкою у ключі
Ключ з помилкою: гелолисоборотней
ронятноеделокуль
ууурунасильновчел
пвеканевоткнешьв
прдусиэтудовольн
пгрустнуюистинуз
оалинаверноелучш
жчемгдебытонибыл
пвмирекультурнос
уьпреждевсегоуси
миеиежелионосызм
I розшифрованого: 0.05358
PS C:\CRYPTOLAB2>
```

Прогноз виявився повністю правильним, у кожному рядку спотворено лише першу літеру, тоді як усі інші 15 стовпчиків зберегли ідеальну читабельність .

При помилці в ключі I незначно знизився до 0.05358. Це пояснюється тим, що 6.25%символів (один стовпчик) перетворилися на «шум» із хибним зсувом, що трохи згладило загальний частотний профіль.

Висновок: Помилка в одному символі ключа є строго локальною. Вона не руйнує загальну структуру відкритого тексту і дозволяє аналітику легко прочитати 93.75% матеріалу та відновити зіпсовану літеру за змістом.

Питання для аналізу результатів

1. Як змінювався індекс відповідності шифртексту при використанні ключів різної довжини? Поясніть отриманий результат.

Індекс відповідності - це статистичний показник, що визначає ймовірність того, що дві випадково обрані літери із зашифрованого тексту виявляться однаковими. Зі збільшенням довжини ключа індекс загалом спадає: від 0.04390 при r = 2 до 0.03314 при r = 12 (тобто чим довший ключ, тим ближче індекс до значення для випадкового тексту (1/32)).

Це пояснюється тим, що шифр Віженера з ключем довжини r фактично складається з r шифрів Цезаря з різними зсувами, які застосовуються до літер відкритого тексту по черзі. Кожен окремий шифр Цезаря зберігає частоти літер (змінюється лише те, якій літері вони належать), але піки частот різних блоків припадають на різні літери. У шифртексті в цілому вони накладаються один на одного, і розподіл літер вирівнюється. Чим довший ключ, тим менша частка пар літер, що належать одному блоку (≈ 1/r), і тим більша частка пар літер із різних блоків, які збігаються лише з імовірністю ≈ 1/32. Тому індекс відповідності прямує до значення для рівноймовірного алфавіту.

2. За якими ознаками було визначено найбільш імовірну довжину невідомого ключа?

Найбільш імовірну довжину ключа r = 16 було визначено за двома основними статистичними показниками: середнім індексом відповідності та автокореляцією. При r = 16 середній індекс відповідності сягнув значення 0.05540, що практично збігається з теоретичним індексом відповідності відкритого тексту російської мови (0.0553). Це підтвердило, що кожен із 16 блоків зашифрований одноалфавітним зсувом. Крім того, статистика збігів автокореляції D_16 показала різкий сплеск до 302 при фоновому рівні шуму 180–210. Це означає, що символи, розміщені на відстані 16 позицій один від одного, зашифровані однією і тією ж літерою ключа.

3. Які альтернативні кандидати на довжину ключа ви досліджували, чому обрали саме їх та які результати отримали?

Як альтернативні кандидати ми досліджували значення r = 4, 8 та 24. Вони були обрані тому, що є дільниками (4 і 8) або мають спільний дільник (24) зі справжнім періодом 16, через що на них виникають так звані гармонійні сплески. Для r = 8 та r = 24 індекс відповідності підвищився до значення близько 0.0449, що вище за випадковий шум (0.03125), але суттєво нижче за норму мови (0.055). Для r = 4 він становив близько 0.0398. При цьому значення автокореляції (D_8 = 189, D_24 = 173, D_4 = 180) залишалися на рівні звичайного фонового шуму і не давали сплесків.

Усі ці кандидати було відхилено, оскільки їхні підвищені показники індексу відповідності є лише математичним наслідком кратності справжньому періоду r = 16.

4. Яка статистична інформація використовувалася для відновлення окремих символів ключа?

Для відновлення літер ключа ми використовували частоти літер у кожному з 16 блоків шифртексту (блок - літери, зашифровані однією і тією ж літерою ключа), зіставлені з частотами літер російської мови (зокрема, найімовірнішою літерою “о” та другою по частоті - літерою “е”).
Для кожного блоку ми підрахували кількість появ кожної літери і виділили 5 найчастіших. Найчастішу літеру блоку y* зіставляли з найімовірнішою літерою мови x* = «о», і літеру ключа знаходили за формулою $k_j = (y^* - x^*) \bmod 32$.
Для деяких блоків такий підхід дав незмістовний текст, тому для них як x* ми взяли наступну за імовірністю літеру «е».

5. Як можна перевірити правильність відновленого ключа, не знаючи початкового відкритого тексту?

По-перше, розшифрований знайденим ключем текст має бути читабельним, не має бути випадкового набору букв, які не складаються в слова. По-друге, можна перевірити індекс відповідності розшифрованого тексту, він має бути близьким або збігатись з теоретичним значеннням для російської мови. По-третє, частоти літер мають відповідати російській мові, найчастішими мають бути «о», «е», «а», «и», «н», «т», а літери «ф», «щ», «ъ», «э» мають бути рідкісними. Ну і сам ключ має бути змістовним (як мінімум в наших роботах).

6. Як змінився результат розшифрування після внесення помилки в один символ ключа? Поясніть характер і періодичність змін.

Після внесення помилки в перший символ ключа (заміна «д» на «г») у кожному рядку розшифрованого тексту спотворилася лише перша літера. Помилка має строго локальний характер. Зіпсувалося рівно 6.25% тексту (1/16), а решта 93.75% залишилися повністю читабельними. Спотворення повторюються з періодом r = 16, тобто зіпсувався лише перший стовпчик (символи з індексами i mod 16 = 0). Індекс відповідності при цьому незначно знизився з 0.05389 до 0.05358 через те, що один стовпчик перетворився на шум із хибним зсувом і трохи згладив загальний частотний профіль.

7. Які етапи проведеного криптоаналізу найбільше залежать від обсягу доступного шифртексту? Чому?

Найбільше від обсягу доступного шифртексту залежить відновлення літер ключа.
Кожна літера ключа визначається за частотами літер у своєму блоці, а довжина блоку у нашому випадку - лише 350–351 літера. Частота літери, оцінена за вибіркою такого

розміру, має похибку, яка приблизно дорівнює різниці між імовірностями «о» (≈ 0,11) та «е» (≈ 0,085), тобто найімовірніших літер російської мови. Тому у нас так сталось, що в частині блоків найчастішою виявляється не «о», а «е». З більшим шифртекстом блоки були б довшими, піки виразнішими, і помилок було б менше.

Окрім відновлення літер ключа, від обсягу доступного шифртексту критично залежить етап визначення довжини ключа. Щоб обчислити середній індекс відповідності та автокореляцію, текст ділиться на r стовпчиків. Якщо шифртекст занадто короткий, вибірка в кожному стовпчику виходить замалою, частоти літер починають сильно флуктувати, а сплеск автокореляції зливається з фоновим шумом. У такому разі відрізнити справжній період (r = 16) від шуму або хибних кандидатів (r = 8) виявиться статистично неможливо.

8. Які результати ваших експериментів найкраще демонструють слабкість шифру Віженера з періодичним ключем?

Слабкість шифру Віженера найкраще демонструють три основні результати. По-перше, це можливість виявлення періоду ключа за допомогою статистики: автокореляція показала чіткий пік D_16 = 302, а середній індекс відповідності піднявся до мовної норми 0.05540, що дозволило легко знайти довжину ключа r = 16 без знання відкритого тексту. По-друге, це зведення поліалфавітного шифру до простих шифрів Цезаря. Після встановлення r = 16 складний шифр розпався на 16 окремих блоків, кожен з яких є звичайним одноалфавітним шифром і зламується стандартним частотним аналізом. По-третє, це відсутність розмноження помилки. Експеримент з помилкою показав, що спотворення однієї літери ключа псує лише 6.25% тексту, залишаючи 93.75% абсолютно читабельними, що дозволяє легко відновити неправильні літери ключа за змістом контексту.